ATIVIDADE 2 - CRIPTOGRAFIA RSA

Parte 2

Aluno: João Pedro De Rodrigues Alves
Matrícula: 2312130223
Disciplina: Estruturas Matemáticas para Computação
Turno: Matutino

1. Objetivo

Nesta atividade foi desenvolvido um programa em Python para demonstrar o funcionamento simplificado da criptografia RSA. O programa trabalha com números primos, calcula os parâmetros necessários para gerar as chaves pública e privada, realiza a cifragem e a decifragem de um número inteiro e demonstra a aplicação dos Teoremas de Euler e Fermat.

2. Primalidade e parâmetros RSA

O programa solicita dois números inteiros, p e q, e verifica se são primos e diferentes. A verificação de primalidade é feita por tentativas de divisão. Em seguida, são calculados o módulo RSA (n) e a função totiente de Euler (phi(n)):

$$n = p * q$$
$$phi(n) = (p - 1) * (q - 1)$$

Depois, o usuário informa o expoente público e. O programa verifica se ele é maior que 1, menor que phi(n) e coprimo com phi(n), ou seja, se MDC(e, phi(n)) = 1.

3. Construção das chaves

O expoente privado d é calculado pelo inverso multiplicativo de e módulo phi(n), utilizando um procedimento baseado no Algoritmo de Euclides Estendido. O resultado é conferido pela igualdade e * d mod phi(n) = 1. A chave pública é formada por (e, n), e a chave privada por (d, n). No teste realizado, foram utilizados p = 5, q = 11 e e = 3. Assim, n = 55 e phi(n) = 40. O inverso encontrado foi d = 27, pois 3 * 27 mod 40 = 1. Portanto, a chave pública é (3, 55) e a chave privada é (27, 55).

4. Cifragem e decifragem

O usuário informa um número inteiro M dentro do intervalo de 0 até n - 1. Para cifrar, o programa utiliza a chave pública. Para decifrar, utiliza a chave privada. Os cálculos são:

$$C = M^e \bmod n$$
$$M' = C^d \bmod n$$

A exponenciação modular é realizada por um procedimento próprio, que reduz os resultados módulo n durante os cálculos. No teste, a mensagem original foi M = 7. A cifragem resultou em 7^3 mod 55 = 13. Depois, a decifragem resultou em 13^27 mod 55 = 7. Assim, o programa recuperou corretamente a mensagem original.

5. Teoremas de Euler e Fermat

Na última parte, o programa demonstra os teoremas utilizando a função de exponenciação modular. Para o Teorema de Euler, verifica se a base e o módulo são coprimos e calcula a função totiente do módulo informado. Quando MDC(a, m) = 1, vale:

$$a^{phi(m)} \bmod m = 1$$

No teste realizado, foram utilizados a = 2 e m = 5. Como phi(5) = 4, o cálculo foi 2^4 mod 5 = 1. Para o Pequeno Teorema de Fermat, o programa solicita um módulo primo p e uma base que não seja divisível por p. Nessas condições, verifica:

$$a^{(p - 1)} \bmod p = 1$$

No teste, foram utilizados p = 7 e a = 3. O resultado foi 3^6 mod 7 = 1, confirmando a aplicação do teorema.

6. Testes realizados

Etapa	Valores utilizados	Resultado
Parâmetros RSA	p = 5; q = 11; e = 3	n = 55; phi(n) = 40
Construção das chaves	e = 3; phi(n) = 40	d = 27; verificação = 1
Cifragem e decifragem	M = 7	C = 13; mensagem recuperada = 7
Teorema de Euler	a = 2; m = 5	2^4 mod 5 = 1
Teorema de Fermat	a = 3; p = 7	3^6 mod 7 = 1

7. Conclusão

Os testes mostraram que o programa calculou corretamente os parâmetros do RSA, encontrou as chaves pública e privada e recuperou a mensagem após a decifragem. Também foram verificadas as relações apresentadas pelos Teoremas de Euler e Fermat. Dessa forma, as quatro partes da atividade foram executadas com os exemplos utilizados.